

DDA5001 Machine Learning

Transformer (Part I)

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



“Attention is All You Need”

- ▶ **Transformer** is one special but one the most important architectures of deep learning.
- ▶ People come to know it widely because of the following paper:

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez*[†] University of Toronto aidan@cs.toronto.edu	Lukasz Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin*[‡] illia.polosukhin@gmail.com			

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Transformer is The Foundation of Language Models

“The transformer, today’s dominant AI architecture, ...has gone on to revolutionize the field of AI over the past half-decade ...”

— Rob Toews, Contributor. Sep. 3, 2023. Forbes Magazine.



- Contemporary big AI models (language models) mostly use transformer architecture.

Agenda

Attention

Attention in Transformer

- ▶ Transformer is inherently designed for natural language processing, but now is also widely used for vision tasks.
- ▶ The key ingredient of a transformer is **attention**, which captures the **semantic structure** of a sentence.

Before introducing the mechanism of attention, we first introduce some basic notions of transformer.

Basic Notions

- ▶ The input data to a transformer: $\{\mathbf{x}_i\}$. Each $\mathbf{x}_i \in \mathbb{R}^d$ is called a token.

Examples:

- ▶ For a sentence like

It is raining.

A token can be a word in this sentence such as “It”. It is transformed into a vector $\mathbf{x}_i$ by embedding.

- ▶ For an image, a token can be a patch.

One can regard $\mathbf{x}_i$ as the usual training data point. We call it token instead of training data. Consequently, x_{ij} is the j -th feature of token $\mathbf{x}_i$.

Basic Notions

- ▶ The input data to a transformer: $\{\mathbf{x}_i\}$. Each $\mathbf{x}_i \in \mathbb{R}^d$ is called a **token**.

Examples:

- ▶ For a sentence like

It is raining.

A token can be a word in this sentence such as “It”. It is transformed into a vector $\mathbf{x}_i$ by **embedding**.

- ▶ For an image, a token can be a patch.

One can regard $\mathbf{x}_i$ as the usual training data point. We call it token instead of training data. Consequently, x_{ij} is the **j -th feature** of token $\mathbf{x}_i$.

In transformers, the data matrix $\mathbf{X}$ is constructed in a row-wise manner:

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\top \in \mathbb{R}^{n \times d}.$$

Basically, you can regard $\mathbf{X}$ as one sentence or one paragraph.

Attention: A Linear Combination

Core idea of attention: Transferring the set of input tokens $\{x_1, \dots, x_n\}$ to the transformed tokens $\{y_1, \dots, y_n\}$, so that $\{y_i\}$ contains richer **semantic** structure.

To perform such a transform, we impose that y_i should **not only** depend on x_i but **the whole** set of input tokens $\{x_i\}$ through a **linear relationship**:

$$y_i = \sum_{j=1}^n a_{ij} x_j.$$

Here, $\{a_{ij}\}$ are called **attention coefficients**. We often impose a simplex structure to $\{a_{ij}\}$, i.e.,

$$a_{ij} \geq 0 \ (\forall i, j), \quad \sum_{j=1}^n a_{ij} = 1 \ (\forall i).$$

$\rightsquigarrow y_i$ is calculated by all $\{x_j\}$, and their importance is based on a_{ij} .

Why Attention?

- ▶ It is easy to see that attention is to **mix all** the token information $\{x_1, \dots, x_n\}$ to generate y_i .
- ▶ This is because one word in a sentence is not fully meaningful in isolation. We often need to understand the whole semantic meaning, and then understand the meaning of a word (token).

Why Attention?

- ▶ It is easy to see that attention is to **mix all** the token information $\{x_1, \dots, x_n\}$ to generate y_i .
- ▶ This is because one word in a sentence is not fully meaningful in isolation. We often need to understand the whole semantic meaning, and then understand the meaning of a word (token).

Let us consider the following two sentences

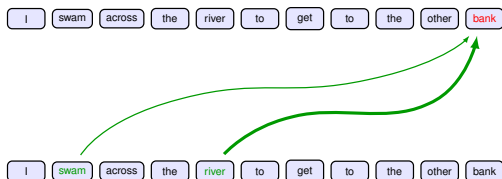
I swam across the river to get to the other **bank**.
I walked across the road to get cash from the **bank**.

Suppose the the word “bank” is a token.

- ▶ It is the same word “bank” in both two sentences. Do they have the same meaning? Obvious not.
- ▶ We need to put the word in the context of the whole sentence, i.e., **other words (tokens) also contribute to “bank”**.

Why Attention?

Hence, attention is to represent the relationship between tokens.



Observations:

- ▶ All tokens are related to each other.
 - ▶ Some tokens are **more important** to interpret “bank”.
- ⇒ Justifies why we need attention mechanism to mix the info of all tokens.

▪

Self-Attention

Self-Attention

However, it is nontrivial to calculate the attention coefficients a_{ij} .

- ▶ Since we need to understand the relationship between different tokens.
- ▶ We also want to determine the importance of the pairwise relationship. **Stronger relation should have more similarity.**

Self-Attention

However, it is nontrivial to calculate the attention coefficients a_{ij} .

- ▶ Since we need to understand the relationship between different tokens.
- ▶ We also want to determine the importance of the pairwise relationship. **Stronger relation should have more similarity.**

This motivates us to compute the similarity using inner product $\mathbf{x}_i^\top \mathbf{x}_j$. Then, the attention coefficients can be constructed using **softmax**:

$$a_{ij} = \frac{\exp(\mathbf{x}_i^\top \mathbf{x}_j)}{\sum_{j'=1}^n \exp(\mathbf{x}_i^\top \mathbf{x}_{j'})}.$$

Such a computed $\{a_{ij}\}$ will satisfy $a_{ij} \geq 0$ ($\forall i, j$), $\sum_{j=1}^n a_{ij} = 1$ ($\forall i$).

- ▶ If for token $\mathbf{x}_i$, we have $a_{ij} > a_{ij'}$ for all other $j' \neq j$, it means $\mathbf{x}_{j'}$ is the most important token in the sentence in terms of interpreting token $\mathbf{x}_i$.
- ▶ This way of determining attention coefficient using tokens themselves is called **self-attention**.

Self-Attention in Matrix Notation

If we stack the output vectors $\{\mathbf{y}_1, \dots, \mathbf{y}_n\}$ to be an output matrix $\mathbf{Y}$ as

$$\mathbf{Y} = [\mathbf{y}_1, \dots, \mathbf{y}_n]^\top.$$

Then, we can have the following matrix form of self-attention

$$\mathbf{Y} = \text{softmax}(\mathbf{X}\mathbf{X}^\top)\mathbf{X}.$$

- ▶ Here, $\text{softmax}(\mathbf{L})$ is first taking exponential to all entries of $\mathbf{L}$.
- ▶ Then, it normalizes each row to sum to one.

Namely,

$$[\text{softmax}(\mathbf{X}\mathbf{X}^\top)]_{ij} = a_{ij}.$$

Each row of $\mathbf{Y}$ is $\mathbf{y}_i^\top$. Each row of $\text{softmax}(\mathbf{X}\mathbf{X}^\top)\mathbf{X}$ is $\sum_{j=1}^n a_{ij}\mathbf{x}_j^\top$.

Trainable Self-Attention

- The attention mechanism is insightful for capturing semantic information. However, the derived self-attention is **fixed** and has **no trainable parameters**.

We have three data matrix $\mathbf{X}$ in the self-attention formula. Indeed, we can manually add trainable parameters to each data matrix as

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}, \mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}, \mathbf{V} = \mathbf{X}\mathbf{W}^{(v)}.$$

Here, $\mathbf{W}^{(q)} \in \mathbb{R}^{d \times d_q}$, $\mathbf{W}^{(k)} \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}^{(v)} \in \mathbb{R}^{d \times d_v}$ are the trainable network parameters.

Trainable Self-Attention

- The attention mechanism is insightful for capturing semantic information. However, the derived self-attention is **fixed** and has **no trainable parameters**.

We have three data matrix $\mathbf{X}$ in the self-attention formula. Indeed, we can manually add trainable parameters to each data matrix as

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}, \mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}, \mathbf{V} = \mathbf{X}\mathbf{W}^{(v)}.$$

Here, $\mathbf{W}^{(q)} \in \mathbb{R}^{d \times d_q}$, $\mathbf{W}^{(k)} \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}^{(v)} \in \mathbb{R}^{d \times d_v}$ are the trainable network parameters.

In this way, the self-attention is generalized to

$$\mathbf{Y} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}.$$

In transformer, $\mathbf{Q}$ is called **query**, $\mathbf{K}$ is called **key**, and $\mathbf{V}$ is called **value**. Note that $d_q = d_k$ must be ensured. One typical choice is

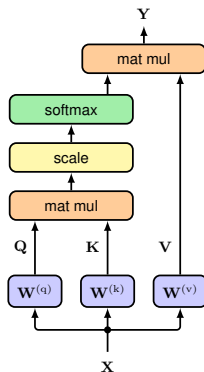
$$d_q = d_k = d_v = d.$$

Scaled Self-Attention

We often re-scale the attention so that gradients of the network will be more stable, giving

$$Y = \text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Illustration:



▪

Multi-head Attention

Multi-head Attention

- ▶ There might be different patterns of attention. Some patterns may be related to tense, some may be related to vocabulary, etc.
- ▶ If we just use one attention, it may learn an **averaged** information of these multiple patterns.
- ▶ Thus, we need multiple attention heads to learn these multiple patterns. Each attention head has its own query, key, value trainable parameters. $\rightsquigarrow$ **multi-head attention**.

Suppose we have H heads defined as

$$\mathbf{H}_h = \text{attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h), \quad h = 1, \dots, H.$$

With

$$\mathbf{Q}_h = \mathbf{XW}_h^{(q)}, \quad \mathbf{K} = \mathbf{XW}_h^{(k)}, \quad \mathbf{V} = \mathbf{XW}_h^{(v)}.$$

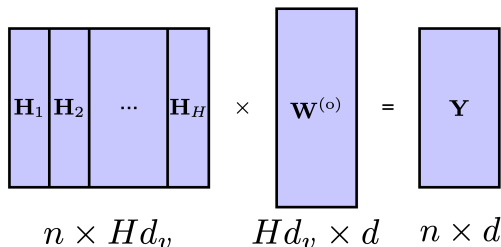
If each $\mathbf{W}_h^{(v)}$ has dimension $d \times d_v$, then each $\mathbf{H}_h$ has dimension $n \times d_v$.

Multi-head Attention

Then, we can introduce a mixing trainable matrix $\mathbf{W}^{(o)}$ to mix all the information as

$$\mathbf{Y}(\mathbf{X}) = [\mathbf{H}_1, \dots, \mathbf{H}_H] \mathbf{W}^{(o)}.$$

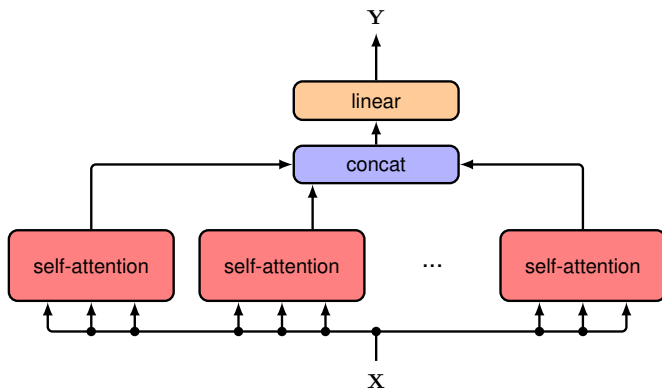
The dimension of the multi-head attention process is illustrated as


$$\begin{matrix} \begin{matrix} \mathbf{H}_1 & \mathbf{H}_2 & \dots & \mathbf{H}_H \end{matrix} \\ n \times Hd_v \end{matrix} \times \begin{matrix} \mathbf{W}^{(o)} \\ Hd_v \times d \end{matrix} = \begin{matrix} \mathbf{Y} \\ n \times d \end{matrix}$$

Since we want to distribute the total dimension d evenly across all the H heads, we typically choose

$$Hd_v = d.$$

Illustration of Multi-head Attention



▪

Attention Masks

Attention Masks: Motivation

In attention, every token could attend to every other token. However:

- ▶ Some positions are **invalid** (e.g., padding) and must not influence the result.
- ▶ Autoregressive decoding must be **causal** (no information leakage from the future).

Examples:

- ▶ **Padding** sequences with different lengths to the same length (for efficient training):

I swam across the river to get to the other bank .

It is raining . pad pad pad pad pad pad pad pad

The padding tokens are **not** real signals.

- ▶ The second case is **causality**. Self-attention at position t can attend to the future tokens $x_{>t}$ as we will input the whole sequence. Then, It will minimize loss by **copying** signals from the future rather than learning real conditional structure.

⇒ Attention masks.

Definition of Attention Mask

Masked attention: Introduce a matrix M and re-write attention as

$$\text{attention}(Q, K, V; M) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V,$$

where

$$M_{ij} = \begin{cases} 0, & \text{allowed} \\ -\infty, & \text{blocked} \end{cases}$$

- ▶ An **attention mask** $M \in \mathbb{R}^{n \times n}$ encodes **hard visibility constraints**. It is **not learned**; it is **constructed** from sequence structure (padding, causality, etc).
- ▶ 0 entry means allowed, i.e., allowing i -th token attending to j -th token.
- ▶ $-\infty$ simply ensures $\exp(-\infty) = 0$ in softmax.

Example: Padding Mask

Scenario: Variable-length sequences are batched by padding shorter sequences with a PAD token.

- Define a boolean **padding indicator** vector $\mathbf{p} \in \{0, 1\}^n$:

$$p_j = \begin{cases} 1, & \text{position } j \text{ is padding} \\ 0, & \text{otherwise} \end{cases}$$

Example: Padding Mask

Scenario: Variable-length sequences are batched by padding shorter sequences with a PAD token.

- Define a boolean **padding indicator** vector $\mathbf{p} \in \{0, 1\}^n$:

$$p_j = \begin{cases} 1, & \text{position } j \text{ is padding} \\ 0, & \text{otherwise} \end{cases}$$

- Construct $\mathbf{M}^{\text{pad}} \in \mathbb{R}^{n_q \times n_k}$:

$$\mathbf{M}_{ij}^{\text{pad}} = \begin{cases} -\infty, & p_j = 1 \\ 0, & p_j = 0 \end{cases} \iff \mathbf{M}^{\text{pad}} = \mathbf{1} \mathbf{p}^\top \cdot (-\infty),$$

where $\mathbf{1} \in \mathbb{R}^{n_q}$ is an all-ones vector and $0 \cdot -\infty = 0$.

- Apply:

$$\mathbf{Y} = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} + \mathbf{M}^{\text{pad}} \right) \mathbf{V}.$$

This prevents attention to padded keys/values from any query position i .

Example: Causal Mask

Scenario: Maintain causality, token i must not attend to future token $j > i$.

- ▶ Define a lower-triangular **causal mask** $M^{\text{causal}} \in \mathbb{R}^{n \times n}$:

$$M_{ij}^{\text{causal}} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

- ▶ Apply (combine with padding mask):

$$Y = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M^{\text{causal}} + M^{\text{pad}} \right) V.$$

Then, all bad attentions will be blocked.

▪

Computational Complexity

Computational Complexity of Attention

Consider the just mentioned masked attention.

- ▶ **Score matrix**: $S = QK^T \in \mathbb{R}^{n \times n}$ with cost $O(n^2d)$.
- ▶ Masking + softmax on S : **Store** and softmax $n \times n$ scores with cost $O(n^2)$ (softmax) and memory $O(n^2)$ (score matrix and mask).
- ▶ Weighted sum: $Y = \text{softmax}(S)V$ with cost $O(n^2d)$.

Observations:

- ▶ Overall, computation and memory are both **quadratic** in sequence length n .
- ▶ Attention is **not** in linear complexity. One of the major complexity of Transformers.

↪ Next lecture: Transformer architecture using attention mechanism.

Flash-Attention

- ▶ Flash-attention is a technique to reduce **storage memory** from $O(n^2)$ to $O(nd)$, i.e., nearly linear in n when n is very large.

The main idea:

- ▶ Rewrites attention as a sequence of **smaller block** matrix operations.
- ▶ **Never** materializes **the full** $n \times n$ score matrix in GPU memory.
- ▶ Uses an **online softmax** algorithm to keep numerical stability while processing tiles/blocks.

Complexity of flash-attention:

- ▶ **Compute:** Still $O(n^2d)$ (same as standard attention).
- ▶ **Memory:** Reduced from $O(n^2)$ to roughly $O(nd)$.

For short sequences (i.e., small n), the benefit is modest. Flash-attention is very useful to reduce GPU memory when n is very large.